Assignment 3

Argumentation Engine

1. Build a class that represents an argumentation framework ⟨A, ⇀⟩. It can be used to create a framework object.
2. Write a function which can test if a given set S of arguments is conflict free.
3. Write a function which can test if a given argument α can be defended by a given set S.
4. Write a function (characteriztic function) to identify the set of arguments defended by a given set S.
5. Write a function to check if a given set S is admissible and is a complete extension.
6. Write separate functions that take a given argumentation framework as input and
    (A) Compute a grounded extension
    (B) Compute all the complete extensions
    (C) Compute a list of all extensions (preferred, stable and semi semi-stable).
    (D) Generate all legal (in/out/undec) labellings
    (E) Report the acceptance status of a given argument α (skeptically accepted/ credulously accepted/ rejected).

Test your functions on the examples given in the book.

Strategic argumentation

The second part is on implementing the grounded direct argumentation mechanism to find out when agents gain by hiding arguments. Each agent's *report* is a revealed subset of its arguments A_i. An agent associates its utility to the number of accepted arguments.

1. Write a function (the mechanism) that takes the global attack relation and one revealed argument set per agent, and returns the grounded extension of the graph induced by the union of the revealed arguments.
2. Write a function that enumerates every subset of A_i that agent i could reveal, given fixed *reports* by the others, and returns the *reports* that maximise the agent *i*'s utility.
3. Write a function to check if a given argumentation framework with grounded direct argumentation mechanism satisfies the Theorem 5.3 by verifying if for every agent i revealing *all* of A_i (truthfulness) is the *best* response against every combination of subsets the other agents could reveal.
4. Write a function which checks if a grounded direct argumentation mechanism is strategyproof by verifying that no agent holds two arguments α, β with an odd-length directed path from α to β in the full graph. [This is Theorem 5.4. Note that you need to deal with possible cycles in the graph].

Experiments:
Given 3 agents, generate at least 1000 random *instances*: 6 to 8 arguments, each possible attack present with probability $p \in \{0.15, 0.25, 0.35\}$, arguments split at random among 3 agents. For each *instance*, record whether the mechanism is strategyproof and whether Theorem 5.4's condition holds. Report:

1. The fraction of instances that are strategyproof, for each p.
2. Check how many instances satisfy the condition in Theorem 5.4 yet fail strategyproofness. Note these instances and explain.
3. Are there instances that are strategyproof although the condition fails? Note these instances and check why they are strategy proof?

There are some misprints in the book. In Example 5.10, the utilities of y and z are written with each other's argument sets. In the discussion after Example 5.11, the argument that cancels the indirect self-defeat is α_6, not α_5.

Submission should include — Code, Readme file, Report and a Demonstration